G10 FX Carry Strategy

Problem Journal

Systematic Macro QIS Backtest - Volatility-Targeted Approach

February 27 – March 5, 2026 | Tashfeen Omran

1. Complete Problem Log

Problem 1: Carry Sign Error & USD-Base Pairs Inverted

What Was Wrong

The first version of the strategy showed a total return of -20.82% with a Sharpe of -0.10, a drawdown that started in 2007 and never recovered, and a hit rate of 46.86%. The strategy was persistently on the wrong side of every trade (**funny or sad?**). JPY and CHF were ranked as long candidates and AUD and NZD were ranked as short candidates, the exact opposite of what carry theory predicts.

Root Cause

In data_loader.py, the carry for USD-base pairs (USDJPY, USDCHF, USDCAD, USDNOK, USDSEK) was computed as rate_USD - rate_foreign instead of rate_foreign - rate_USD. Since the FX returns for these pairs were already sign-flipped to represent 'long foreign currency,' the carry formula needed to be uniform (rate_foreign - rate_USD) across all pairs. Using rate_USD - rate_foreign for USD-base pairs made USDJPY carry appear positive (+3.06%) when it should be deeply negative (-3.06%). The strategy was systematically long the worst carry positions.

The Fix

Unified the carry formula: carry = rate_foreign - rate_USD for ALL 9 pairs. The pair_to_foreign_currency mapping was already correct (USDJPY → JPY, etc.). The fix was a one-line change in compute_carry_per_pair() — removing the USD-base pair special case and applying the same formula everywhere.

Problem 2: FX Returns Sign Flip & USD-Base Pairs Not Verified

What Was Wrong

After fixing the carry sign, the diagnostic showed USDJPY returns as -0.048 for Jan–Mar 2019 when JPY was strengthening (interesting observation). A 'long JPY' return should be positive when JPY strengthens. The flip appeared to be missing or double-applied.

Root Cause

The flip was correctly implemented in load_fx_data() using: if pair in USD_BASE_PAIRS: log_returns = -log_returns. However, the diagnostic was misread initially. After careful verification: when USDJPY falls (JPY strengthens), the raw log return is negative; after the flip, it becomes positive, correctly representing 'being long JPY made money.' The diagnostic confirmed the flip was working. The confusion arose from not accounting for whether JPY actually strengthened in that specific period.

The Fix

Added explicit verification script (verify_returns_flip.py) that checks a known historical period where JPY direction is unambiguous and confirms the sign of the returned series. The flip code itself required no change.

Problem 3: Vol-Targeting Lookahead Bias & Portfolio Vol Not Lagged

What Was Wrong

The scale_to_target_vol() function computed rolling portfolio volatility and immediately used it to scale the same day's weights (**rookie mistake**).

Root Cause

compute_portfolio_volatility() returned a rolling 63-day vol estimate ending on day T. This was then used to compute the scalar for day T's weights. But the weight for day T should only use information available before day T i.e., vol through day T-1.

The Fix

Added port_vol = port_vol.shift(1) immediately after the portfolio vol computation in scale_to_target_vol(). One line, applied before the scalar = target_vol / port_vol calculation.

Why This Choice

The 1-day lag is the standard approach, I guess? It mirrors what a real trader does which is to compute yesterday's realised vol at market close and use it to set today's position size before the market opens. Any shorter lag would use information that isn't yet available at the time of trading. The 63-day window was chosen because it is long enough to smooth out daily noise but short enough to respond to regime changes within a quarter which is the standard window used in institutional carry products.

Problem 4: Vol-Targeting Scalar Floor Missing & Strategy Crushed During High-Vol Regimes

What Was Wrong

The leverage chart showed the vol scalar dropping to 0.45x during the 2008 GFC and remaining below 1.0x for extended periods in 2009–2012 and around 2020. With no floor on the scalar, the strategy was operating at less than half its intended size during volatile periods. Combined with the carry unwind and thus losses were amplified and recoveries were suppressed because the book was too small to benefit when conditions improved.

Root Cause

scale_to_target_vol() had scalar = scalar.clip(upper=max_leverage) but no lower bound. When realized portfolio vol spiked to 20%+ during the GFC, the scalar of 0.10/0.20 = 0.50x was reasonable, but in some extreme periods it dropped to 0.45x or lower. This is mechanically correct behavior for vol-targeting, but institutionally, a floor prevents the book from becoming negligibly small.

The Fix

Added scalar = scalar.clip(lower=0.5, upper=max_leverage) — a 0.5x floor alongside the 2.0x ceiling.

Problem 5: Row-by-Row Signal Loop Thus Slow and Error-Prone

What Was Wrong

compute_raw_signals() in signals.py iterated over every date using a Python for-loop (for date in carry.index), then assigned signals row by row. For 6,206 dates this was extremely slow and also created risk of subtle NaN propagation at edge cases.

Root Cause

The original implementation used carry.loc[date] inside a loop — a pattern that is correct but approximately 100x slower than vectorised pandas operations. It also required manual NaN handling at each row.

The Fix

Replaced the entire loop with three vectorised lines: ranks = carry.rank(axis=1, ascending=False, method='first'), then raw_signals[ranks <= 3] = 1.0, raw_signals[ranks > 6] = -1.0, with everything else defaulting to 0.0. Functionally identical but runs in milliseconds?

Problem 6: Correlation to SPY Showed 0.40 & Suspected Levels vs Returns Bug

What Was Wrong

The correlation heatmap showed 0.40 between the strategy and SPY. Initial suspicion was that cumulative wealth levels (both trending upward) were being correlated instead of daily returns and that would produce a spuriously high correlation.

Root Cause

My marvelous investigation showed the correlation_to_benchmark() function in metrics.py was correctly using daily returns (strategy_aligned.corr(benchmark_aligned)). The 0.40 correlation was computed on daily return series, not on cumulative levels. Additionally, there was a date-alignment inconsistency: the benchmark was being aligned outside run_backtest() using a separate benchmark_aligned series, while the results DataFrame also contained a pre-aligned benchmark_return column from inside run_backtest(). Two slightly different date ranges were in use.

The Fix

Standardized the benchmark alignment: compute_all_metrics() and the correlation heatmap both now use results['benchmark_return'] — the series already aligned inside run_backtest() via date intersection. One source of truth.

What happened

The 0.40 correlation is real given the G10 carry has positive unconditional correlation to global risk appetite. During 2008, 2015, 2020, and 2022 which were all global risk-off episodes, both carry positions (long AUD/NZD, short JPY/CHF) and equity markets fell simultaneously.

Problem 7: Transaction Cost Drag Nightmare 0.98% Per Year Killing Recoveries

What Was Wrong

The gross vs net annual return table showed TC drag of 0.70% or around1.69% per year with a mean of 0.98%. Institutional G10 FX transaction costs should be under 0.30% annually for a monthly-rebalancing carry strategy and the excess drag was silently erasing recovery periods with years like 2009 (+15.48% gross) were still positive net, but the compounding of 1%+ annual drag over 23 years added roughly 23% of cumulative return loss on top of genuine strategy underperformance.

Root Cause

There were two compounding issues. First: the vol scalar was recalculated every single day using a rolling window, which caused weights to change by small amounts every day even when signals were unchanged generating daily phantom turnover. With 2 bps cost per unit of weight change applied to this daily drift the annual drag compounded to ~1%. Second: the signal ranking was updating daily because forward-filled monthly interest rates had micro floating-point differences day to day after the ffill operation, causing carry.rank() to produce slightly different rankings on consecutive days even when the underlying data hadn't changed.

The Fix

Two fixes applied together. Fix 1: resample carry to month-end frequency before ranking (carry_monthly = carry.resample('ME').last()), assign signals at monthly frequency, then forward-fill back to daily. This ensures rankings only change when interest rate data actually changes. Result: 37 signal change days over 6,206 trading days. Fix 2: charge transaction costs only on days where signals actually change (signal_changed = signals.diff().abs().sum(axis=1) > 0), applying 2 bps on turnover only on those days. **Mean annual TC drag fell from 0.98% to 0.03%.**

Why This Choice

Monthly resampling of carry rankings is the theoretically correct approach. Interest rate data is published monthly and carry rankings should update monthly. Daily forward-filling of monthly data does not create new information and charging costs against that daily repetition is economically wrong as I am not actually trading on those days. The 2 bps cost assumption remains realistic for institutional G10 FX bid-ask spreads (~0.5–1 bps per side for major pairs). Charging it only on the 37 actual rebalance days gives ~0.03% annual drag, which is the correct institutional expectation.

Problem 8: Regime Filter That I Attempted and Abandoned

What Was Wrong

Three versions of a regime filter were implemented, attempting to only run the carry strategy when rate differentials were sufficiently wide (i.e., when there was actually something to earn). The goal was to sidestep the ZIRP period (2009–2021) when G10 differentials compressed to near-zero.

Root Cause

Version 1 used the mean absolute carry across all 9 pairs as the regime signal (stupid I know). The middle 3 neutral pairs have carry near zero, dragging the average down and keeping the filter almost always off. Version 2 used the correct active-legs-only calculation (average of top 3 long + bottom 3 short carry magnitudes) but didn't calibrate thresholds and at 3% the strategy was invested only 3.4% of the time (hit rate 1.38%, kurtosis 177, mathematical artifacts of near-zero observations). Version 3 at 1% threshold gave 81% invested time, 1.5% threshold gave 56% sensible fractions, but performance was still worse than no filter: Total return -45.85% vs -29.35% without regime filter.

The Fix

The regime filter was removed from the final version. The unfiltered strategy with correct TC methodology (0.03% drag) produces better results than any version of the filter tested.

Problem 9: compute_inverse_vol_weights & Still Using Row-by-Row Loop

What Was Wrong

After vectorising compute_raw_signals(), the compute_inverse_vol_weights() function in portfolio.py still used a Python for-loop over every date to compute inverse-volatility weights. This made portfolio construction slow but did not affect correctness.

Root Cause

The loop was left in place because the inverse-vol weighting within separate long and short groups is more complex than simple ranking as it requires conditional logic (compute weights separately for the 3 longs and 3 shorts) that is harder to vectorize cleanly (**I admit that I'm not that good**). Vectorizing it incorrectly would be a worse outcome than leaving it slow.

The Fix

The loop was retained with a note in the docstring. Performance is acceptable (~5 seconds for 6,206 dates on a standard laptop). If the dataset were larger or the strategy needed real-time computation, this would be the first function to vectorize.

---

2. What Is Every Parameter & Why It Was Chosen

2.1 Universe: 9 G10 Currency Pairs

EURUSD, USDJPY, GBPUSD, AUDUSD, NZDUSD, USDCAD, USDCHF, USDNOK, USDSEK.

The G10 currencies are the 10 most liquid currencies in the world (USD, EUR, JPY, GBP, AUD, NZD, CAD, CHF, NOK, SEK). With USD as the funding base, we get 9 pairs. These pairs have deep and liquid markets with minimal bid-ask spreads and no capital controls which is essential for a carry strategy where you are holding positions for weeks to months. EM currencies could've been used but were excluded because adding them would require different data sources and I genuinely were running out of time, but also more complex cost assumptions and changes to the signal construction logic. The pure G10 implementation is the academic baseline from Lustig and Verdelhan (2007).

2.2 Tercile Split: Top 3 Long, Middle 3 Neutral, Bottom 3 Short

With 9 pairs, dividing into thirds gives 3 longs, 3 neutrals, 3 shorts makes a standard construction in the academic literature. The middle 3 are excluded because their carry is near the cross-sectional median and they provide neither strong carry income nor strong funding benefit. Including them would dilute the signal and increase turnover for minimal return benefit. Going to top-4/bottom-4 would give more diversification but would include pairs with weaker carry signals, I'm yet to try this.

2.3 Vol Window: 63 Days

The 63-day rolling window for realized volatility is the standard institutional choice for a quarterly risk horizon from several academic literature. It is long enough to smooth out day-to-day noise and short enough to respond to regime changes within a quarter. A shorter window (21 days) would be noisier and cause more frequent leverage adjustments which would increase our turnover problem. A longer window (126 days) would respond too slowly to volatility spikes like 2008 and underprice risk during carry crashes.

2.4 Target Volatility: 10%

10% annualized volatility is the standard (?) QIS product specification for a carry strategy. When banks sell carry products to clients, they are sold as '10% vol' or '7% vol' products (apparently 7% is more popular as that's what I noticed from several other institutional QIS strategies) this allows clients to compare across strategies on a common risk basis. Using 10% means our realized vol (9.6%) is very close to target, validating the vol-targeting mechanism. The vol-targeting also makes the Sharpe ratio meaningful because vol is controlled, the Sharpe reflects pure signal quality rather than leverage effects.

2.5 Max Leverage: 2.0x

The scalar is capped at 2.0x to prevent excessive leverage during low-volatility regimes. During ZIRP the FX volatility compressed significantly and without a cap, the scalar would have pushed leverage to 3–4x (it happened first and then I iterated). At 2.0x, the maximum position size is twice the base case and the 0.5x floor prevents the book from being crushed below half-size during extreme vol events.

2.6 Transaction Costs: 2 bps on Signal-Change Days Only

2 basis points per unit of weight change is a realistic institutional cost for major G10 currency pairs. EUR/USD bid-ask spreads are typically 0.3–0.5 bps for institutional size and other G10 pairs are wider at 1-3 bps. 2 bps as a blended average is slightly conservative but defensible and crucially, costs are only charged on the 37 days per 6,206 where the carry ranking changes not on days where only the vol scalar drifts (that's before the edits). This gives a mean annual TC drag of 0.03%, which is the correct institutional expectation for a monthly-rebalancing carry strategy.

2.7 Interest Rate Data: 3-Month Interbank Rates

The 3-month interbank rate is the standard benchmark for carry strategy construction. It is more directly linked to FX forward pricing than overnight rates and in the academic literature (Lustig, Verdelhan 2007; Burnside et al. 2011), 3-month rates are the standard instrument. The FRED series used (IR3TIB01XXM156N format) are the BIS/OECD standardized interbank rates which makes them directly comparable across countries.

3. My Strat Vs Similar Institutional

Vs. Morgan Stanley MSFX G10 Risk Adjusted Carry Basket Index

The Morgan Stanley G10 Risk Adjusted Carry Basket Index aims to capture the forward rate bias (carry) in the same G10 universe. Differences from our strategy:

- Proprietary risk-adjusted weights: MSFX weights are “computed monthly by Morgan Stanley using its proprietary model” and are not disclosed. “Risk adjusted” implies volatility- or risk-based weighting rather than our transparent inverse-vol scheme. We use a standard academic-style inverse-vol allocation within long and short buckets.
- Target volatility and rebalance: The index targets 15% volatility (we use 10%) and rebalances on the first GBP business day preceding the second Monday of each month. Our vol target and monthly signal logic are similar in spirit but not identical.
- Implementation and data: MSFX is built from MSFX single-currency total return indices (WM fix, spot/next, funding), i.e. a production FX index environment. We use spot returns plus FRED 3-month interbank rates and do not use forwards or proprietary data.

Vs. Bloomberg GSAM FX Carry Index

The Bloomberg GSAM FX Carry index (starting at 100 in 2005, ending at ~168 in 2025) appears to outperform our strategy significantly. The gap is explained by three structural differences:

- EM exposure: The GSAM index blends G10 carry with Emerging Market carry and EM currencies maintained 8-15% yields even through ZIRP, which is far wider differentials than anything in G10. This is where most of the post-2008 carry premium lived. Our pure G10 implementation has no EM exposure.
- Implied yield momentum: GSAM uses FX implied yields and momentum dynamics and not just spot-rate-plus-interbank-differential. This hybrid signal incorporates information from the forward curve that our 3-month FRED rate approximation misses.
- Optimized weighting: GSAM uses volatility-based optimization and not simple inverse-vol weighting. Their implementation is a production system with years of refinement.

Vs. S&P Risk Premia FX G10 Carry Index

The S&P Risk Premia FX G10 Carry Index is a rules-based G10 carry index. Main differences:

- Carry definition and data: S&P defines carry as the 3-month yield and for non-USD currencies this is the implied 3M yield from spot, 3M FX forward, and the 3M US T-Bill and not interbank rates. We use FRED 3-month interbank rates and our carry is an approximation to the forward-implied yield.
- Weighting and rebalance: S&P uses tiered weights (50%, 30%, 20% for the top three and −50%, −30%, −20% for the bottom three by carry rank). No equal or inverse-vol and rebalancing is on IMM dates i.e. quarterly. We use inverse-vol weights and monthly signal updates.
- Execution: The index is implemented via quarterly rolling currency forwards (settlement aligned to the next IMM date). We use spot returns plus interest differential and thus I do not explicitly roll forwards.